

Justice League: Time-series Game Player Pattern Detection to Discover Rank-Skill Mismatch

Haerin Kim
Korea University
Seoul, Republic of Korea
lini1634@korea.ac.kr

Sangho Lee
Korea University
Seoul, Republic of Korea
lee35@korea.ac.kr

Ji Young Woo
Soonchunhyang University
Asan, Republic of Korea
jywoo@sch.ac.kr

Huy Kang Kim
Korea University
Seoul, Republic of Korea
cenda@korea.ac.kr

Abstract—When rank and skill do not coincide in competitive games, this might be a sign of issues such as boosting, smurfing, and trolling occur. The fair gaming culture of online gaming is disrupted and offended by cheating like boosting, smurfing, and trolling. The player's play style must be used to determine the rank that appears in account information. In this study, we classified League of Legends' low and high tiers using a sequence-based CNN-LSTM model. Using input perturbation, the model can explain its own importance for certain features. The experimental progress: First, we selected features that show a difference between tiers by an extracted score estimating a cumulative sum graph. Second, we construct the dataset format variously with variable or fixed sequence length, compare performance, and analyze the pros and cons. Finally, we consider the possibility of early detection by measuring performance over game elapsed time. Along with the experiment, a rank classification performance of the model achieved AUC 0.9036 and found that we can distinguish from the 24 minutes after the start of the game. In addition, We derived that *ccReduction* and *MinionsKilled* were the information that had the most influence on skills among various features.

Index Terms—online games, data engineering, deep learning, cheating

I. INTRODUCTION

Cheating leveraging the mismatch between the opponent players can occur because account information does not reflect the actual players' skills. In particular, in the case of a game in which a matchmaking system exists, it is essential to play a fair game because it competes with other players. Representative problems when account information and players' playing skills do not match include boosting, smurfing, and trolling.

Boosting allows a higher-skilled player to log into another player's account to play matches on behalf of the player and 'boost' the player's account to a higher level, a violation of Riot Games' End User License Agreement (EULA). It is common in many competitive games, including League of Legends, Overwatch, and Dota 2, and the scale is quite large. The revenue from boosting for each match is estimated to be approximately € 111,566,660 [1]. Smurfing is an act that intentionally adversely affects low tiers players when a highly skilled player creates a new account. Kou *et al.* [2] describe that the player had negative experiences affected by smurfing. Trolling destroys the game with acts that deliberately hurt our troops. There can be many kinds of trolling. For example, in the in-game chat, players who play games together can be

exhausted by saying insulting words and actions to the other person or repeating the same behavior.



Fig. 1: Summoner's rift where each game of League of Legends occurs [3].

To resolve the problems, we propose tier-group classification, which can evaluate the rank-skill match. It classifies skills based on play patterns. We choose the online game League of Legends, a popular game, to classify the tier-group. League of Legends is a Multiplayer Online Battle Arena (MOBA) game where two teams of 5 players battle. The game's objective is to destroy the opponent's Nexus, the main structure in the game, by collaborating with team members. Each team has to defend on each side of the map fig. 1. Players have designated a role in their team. The roles are *Top Laner*, *Jungler*, *Mid Laner*, *Bot Laner*, and *Support*. Each player picks one champion in the match, considering its lane and team position.

With the large size of user behavior investigation, artificial intelligence is necessary to detect suspicious activities or cheating in online games efficiently. In this paper, we analyze players' play pattern using data analysis and deep learning models. We developed a model to detect rank-skill mismatch using a real dataset of a popular online game, League of Legends. The rank-skill mismatch is a characteristic commonly found in cheating behaviors such as boosting, smurfing, and trolling. The key contributions of this paper are as follows.

- To the best of our knowledge, it is the first attempt to investigate the problem that the account rank and play patterns are mismatched. To address this issue, we propose an experimental design based on a deep learning model using a real dataset of a popular online game, League of Legends.

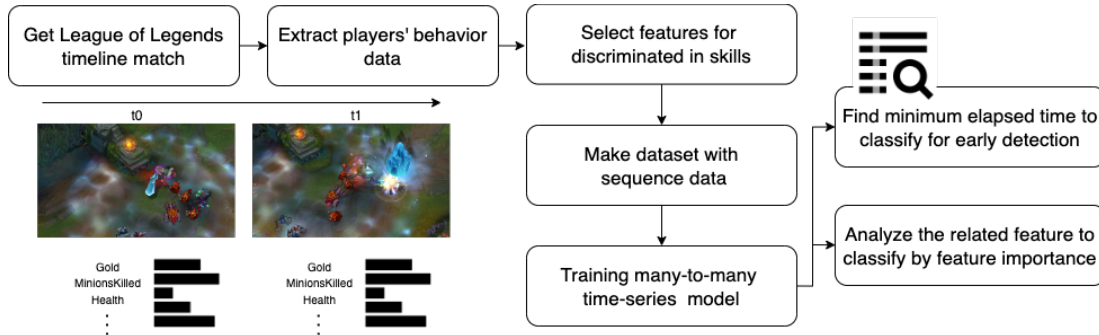


Fig. 2: Overall process to detect the rank-skill mismatch problem.

- We propose a feature elimination method. Many features can cause a load on memory and cause noise in the necessary information. We define and propose a non-traditional library method from feature selection to feature analysis.
- We construct the dataset in various ways depending on whether the sequence length is variable or fixed, including methods for creating a fixed length without data loss.

We elaborate on the process of the tier-group classification shown as fig. 2. First, we describe how to collect League of Legends game data from Riot Games' public APIs, and which part of the dataset was extracted and used as a feature to identify the player's behavior pattern. Then we indicate which criteria were selectively chosen to optimize the number of features. Finally, we suggest the analysis method used to determine the player's behavior pattern.

II. RELATED WORK

Chen *et al.* [4] proposed the RET method, a user identification method, based on the distribution of idle time using data from the game Angel's Love. They evaluated the performance with the player's activity history size and detection time. Kaminsky *et al.* [5] identify users using mouse movements such as clicks and drags. Liu *et al.* [6] predicted the next move with replay data from StarCraft II using machine learning algorithms from the WEKA toolkit. Da Silva and Da Costa-Abreu [7] analyzed biometric information such as keystroke and mouse dynamics to investigate how neural network algorithms show different results in League of Legends. As a way of identifying game players, several studies have been conducted using mouse dynamics among the biometric process [8], [9]. Hooshyar *et al.* [10] summarized a lot of data-driven approach player modeling to predict players' behavior. They categorized it according to data analysis methods and organized detailed algorithms. Liu *et al.* [11] proposed a new algorithm combination of Markov models, state aggregation, and player heuristic search to address a broad range of educational game input but sparsely observed data. Krishnan *et al.* [12] presented Blackout, which is an action model learning (AML) algorithm to generate player models and compare with the existing AML algorithm, FAMA.

III. RANKING SYSTEM

A ranking system in League of Legends has nine tiers, including Iron, Bronze, Silver, Gold, Platinum, Diamond, Master, GrandMaster, and Challenger. Each tier has different ranking distribution, and most players are Iron, Bronze, and Silver rank shown as fig. 3¹. These three tiers account for more than half of the total users. The players' distribution

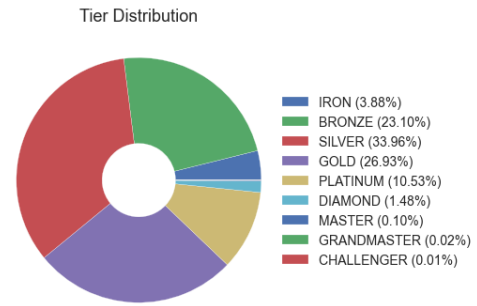


Fig. 3: Tier distribution in League of Legends.

higher than the Gold tier decreases sharply. Above the Platinum rank is generally regarded as a competitive player, and Platinum and Diamond tier account for the most significant proportion of the competitive players in the top tier. The top ranks, Master, GrandMaster, and Challenger, are very few, and Iron to Diamond have detailed steps in the tier, but the steps are not divided from this rank. The matchmaking system assigns players of similar skills to a match. Tier is the most straightforward and most clear metric to distinguish skills. Therefore, we are labeling tiers with the player to classify the skills.

IV. DATA COLLECTION

In order to understand the player's skills as a play pattern of the game, we use a dataset that records behavioral changes over time, such as in the bottom left of fig. 2. Riot Games' League of Legends provides data that collects player logs on a

¹<https://www.op.gg/statistics/tiers>

TABLE I: The number of match data

	Iron	Bronze	Silver	Gold	Platinum	Diamond	Master	GrandMaster	Challenger
2021	38,882	47,325	53,338	47,134	44,977	49,503	54,269	50,386	55,038
2022	5,958	5,988	5,023	5,983	5,973	5,964	1,284	3,500	2,060

minute basis in a game. Thus we collected MATCH-V5² timeline data through Riot Games' public APIs. LEAGUE-V4³, SUMMONER-V4⁴, and MATCH-V5 were used to extract multiple match Ids corresponding to players in a particular tier. First, we select the tier and receive all the league entries with the API of LEAGUE-V4. Second, we use the SUMMONER-V4 API to get detailed summoner information. Finally, we can obtain the match id corresponding to the summoner information; and we get a match timeline from the match id with the API of MATCH-V5. We focus on Solo Rank among various types of games because it is the most popular and can show each player's abilities well. Therefore, only the queue ID of '420' corresponding to solo rank 5x5 was filtered out.⁵ Among the collected data, a total of 440,852 match data played between March 31 and December 30, 2021 were used as training and validation data, and a total of 41,736 match data played in January 2022 were used as test data. table I is the number of match data corresponding to each rank.

V. FEATURE SELECTION

The match timeline is in json format. It is divided into metadata and info forms, including data version, match id, and *puuid* for players playing in ranked games. The info has a frame interval of a minute, matched by frames, game id, and *puuid* by *participantId*. *participantId* is the information used by match timeline as an integer value. Frames have information about *events*, which shows the player's events, and *participantFrames*, providing the player's information every minute. *participantFrames* has values such as champion status, damage status, and total gold, with players participating in the game per minute as keys to *participantId*. *events* has information that shows what behavior the player showed for each event, such as building kill and item purchased. Among them, we extract only *participantFrames* to analyze player behavior information rather than to grasp the center of events within the game. The features are shown in table II.

In order to create a model for classifying tier-group, we need features that differ by tier, which are filtered by ranking the distinctiveness of features and partially eliminating them. We proposed the feature selection method as shown in algorithm 1. For example, a feature called *MinionsKilled* has a value from 1 minute to 25 minutes in the IRON tier, as many as the number of players in that tier. First, we calculate the expected value to determine the difference in features for each tier. Second, we converted normalized values between 0 and 1

²<https://developer.riotgames.com/apis#match-v5>

³<https://developer.riotgames.com/apis#league-v4>

⁴<https://developer.riotgames.com/apis#summoner-v4>

⁵<https://static.developer.riotgames.com/docs/lol/queues.json>

TABLE II: Extracted features (44)

Feature	Description
abilityPower	Stat that increases the effectiveness
armor	Amount that reduces incoming physical damage
armorPen	Stat that the target's armor is ignored when dealing physical damage
armorPenPercent	Percentage of armor penetration
attackDamage	Amount of physical damage
attackSpeed	Attacks per second
bonusArmorPenPercent	Bonus percentage of armor penetration
bonusMagicPenPercent	Bonus percentage of magic penetration
ccReduction	Reduction of crowd control
cooldownReduction	Time stat for re-use of champion abilities
currentGold	Current gold amount
goldPerSecond	Gold per second
health	Health of the champion
healthMax	Maximum health of the champion
healthRegen	Stat that determines the amount of health a champion regenerates
jungleMinionsKilled	Number of jungle minions killed
level	Champion's level
lifesteal	Offensive stat which grants healing damage dealt by basic attacks
magicDamageDone	Magic damage done
magicDamageDoneToChampions	Magic damage done to champions
magicDamageTaken	Magic damage taken
magicPen	Stat that the target's magic resistance is ignored when dealing magic damage
magicPenPercent	Percentage of magic penetration
magicResist	Amount that reduces incoming magic damage
minionsKilled	Number of minions killed
movementSpeed	Stat that represents the rate at which a champion travels across a map
physicalDamageDone	Physical damage done
physicalDamageDoneToChampions	Physical damage done to champions
physicalDamageTaken	Physical damage taken
power	Power of the champion
powerMax	Maximum power of the champion
powerRegen	Regeneration power of the champion
spellVamp	Stat which grants healing equal to a percentage of the ability damage dealt
timeEnemySpentControlled	Time spent to control the enemy
totalDamageDone	Total damage done
totalDamageDoneToChampions	Total damage done to champions
totalDamageTaken	Total damage taken
totalGold	Total gold amount
trueDamageDone	True damage done
trueDamageDoneToChampions	True damage done to champions
trueDamageTaken	True damage taken
x	Horizontal axis position of the champion
xp	Amount of experience points
y	Vertical axis position of the champion

for all tiers into a cumulative sum accumulated for each period. Finally, we calculate the difference between tiers using Dynamic Time Warping (DTW). By averaging those difference values, the differentiation of features was expressed. As a result of the calculation, we examine the performance changes by removing five features containing the lower 10%, nine features containing the lower 20%, 13 features containing the lower 30%, and 18 features containing the lower 40%, and 22 features containing the lower 50%.

VI. TIME-SERIES LENGTH ADJUSTMENT

In this experiment, we assumed Bronze, Silver and Gold are classified into the *lower-tier group*, Master, GrandMaster, and Challenger are classified into the *upper-tier group*. The player's match length is usually distributed in 20 to 40 minutes

Algorithm 1 calculate feature score (T=25)

Input: $timeline_match[9][N][T]$ **Output:** $feature_scores[9]$ **Step 1: Get mean value of every user** $mean_per_tier[9][T]$ **for each** $i \in tier[0, 9 - 1]$ **do** $mean_per_time[T]$ **for each** $t \in time[0, T - 1]$ **do** $sum_1 \leftarrow 0$ **for each** $j \in users[0, N - 1]$ **do** $sum_1 \leftarrow sum_1 + timeline_match[i][j][t]$ **end for** $mean_per_time[t] \leftarrow \frac{sum_1}{N}$ **end for** $mean_per_tier[i] \leftarrow mean_per_time$ **end for****Step 2: Normalization and get cumulative sum** $cumsum[9][T]$ **for each** $i \in tier[0, 9 - 1]$ **do** $sum_feature \leftarrow sum(mean_per_tier[i])$ **if** $sum_feature = 0$ **then** $cumsum[i] \leftarrow [0...]$ **else** $sum_2 \leftarrow 0$ **for each** $t \in time[0, T - 1]$ **do** $sum_2 \leftarrow sum_2 + \frac{mean_per_tier[i][t]}{sum_feature}$ $cumsum[i][t] \leftarrow sum_2$ **end for****end if****end for****Step 3: Calculate the average DTW value for each tier****for each** $i \in [0, 9 - 1]$ **do** $sum_3 \leftarrow 0$ **for each** $j \in [0, 9 - 1]$ **do** $sum_3 \leftarrow sum_3 + dtw(cumsum[i], cumsum[j])$ **end for** $feature_scores[i] \leftarrow \frac{sum_3}{9}$ **end for**

but varies widely from 5 minutes to sometimes more than an hour. Fast processing is possible if the deep learning model can receive such a variable match length as an input. This method is referred to as “raw” in this study. However, performance may change depending on the batch size of the deep learning model. When the match length is fixed, the best performance can be achieved with various batch sizes. In order to keep the length fixed, there is a method of throwing away the shorter one than a certain match length and using the longer one up to the corresponding length. This method is referred to as “cut” in this study. Throwing away data shorter than a certain match length is a data loss. Therefore, data can be transformed and used by increasing the length without throwing it away. This method is referred to as “pool” in this study. Because the pool method requires pretreatment compared to the previous two

methods, it takes a little more time. The pool method stretches the input data by making a space equal to the minimum common multiple of the input length and the length to be fixed and then fills each input value by the fixed length. In addition, the output value is derived by the length of the input length to fix the average inner surface. The process is the same as the algorithm algorithm 2.

Algorithm 2 pool method

Input: $input_array[N]$, $target_size$ **Output:** $output_array[target_size]$ $K \leftarrow target_size$ $LCM \leftarrow lcm(N, K)$ $tmp_array[LCM]$ $L_1 \leftarrow \frac{LCM}{N}$ **for each** $i \in [0, N - 1]$ **do****for each** $j \in [0, L_1 - 1]$ **do** $tmp_array[j] \leftarrow input_array[i]$ **end for****end for** $L_2 \leftarrow \frac{LCM}{K}$ **for each** $i \in [0, K - 1]$ **do** $sum \leftarrow 0$ **for each** $j \in [0, L_2 - 1]$ **do** $sum \leftarrow sum + tmp_array[L_2 * i + j]$ **end for****if** $i = 0$ & $input_array[0] = 0$ **then** $output_array[i] \leftarrow 0$ **else** $output_array[i] \leftarrow \frac{sum}{L_2}$ **end if****end for**

We performed the experiment by applying all of the raw, cut, and pool methods to the training dataset and only raw methods to handle various match lengths to the test dataset. In addition, the advantages and disadvantages of each method are compared and analyzed with performance.

VII. PLAY PATTERN CLASSIFIER

It allows the player’s play pattern can be input every minute to output whether the player corresponds to a high tier at the corresponding time. We use a sequence-based deep learning model to distinguish skills by the player’s behavior patterns over time. It was based on the Long Short-Term Memory (LSTM) layer, which is widely used as a sequence-based model. We constructed the model consisting of the convolution layer, pooling layer, and LSTM layer. The convolution layer extracts local features of the raw time-series data. The LSTM layer learns a sequence of features. In analyzing time-series data in various fields, researchers use a model that combines Convolutional Neural Network (CNN) and LSTM. Wang *et al.* analyzed text data and predicted the valence arousal (VA) ratings by combining the regional CNN, and LSTM [13]. In the energy field, Kim *et al.* proposed a CNN-LSTM network

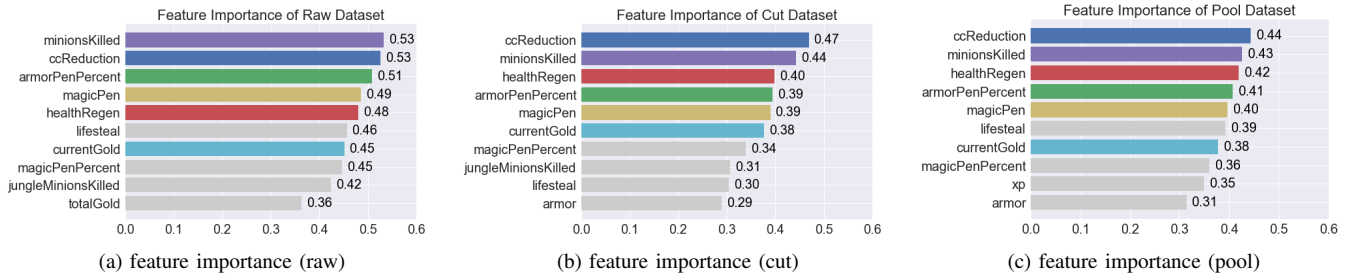


Fig. 4: Visualization of the top 10 features with high importance out of 44 features. For each dataset configuration method, the same features are expressed in the same color, and bright gray is a feature that is not included in all dataset configurations.

to predict housing energy consumption [14]. Livieris *et al.* applied CNN-LSTM model to predict cryptocurrency [15]. CNN allows the extraction of peripheral features between variables, and LSTM is capable of learning long-term dependencies in the sequence data.

TABLE III: Play pattern classification model layer architecture

Layer	Output Shape
Conv1D	(None, None, 512)
BatchNormalization	(None, None, 512)
LSTM	(None, None, 256)
Dropout	(None, None, 256)
LSTM	(None, None, 128)
Dropout	(None, None, 128)
LSTM	(None, None, 96)
Dropout	(None, None, 96)
TimeDistributed	(None, None, 1)

The components and order of each layer are the same as table III. In the output structure, the first None is the batch size, and the second None is the match length. The match length was not specified because the test data had a variable match length. Dropout was applied to each LSTM layer to prevent overfitting the learning process. In order to output the result every minute of a match, the last layer was the time distributed to the dense layer. We use binary cross entropy as a loss function. The model’s learning rate is 0.0005 with RMSprop optimizer, 32 of batch size, and we used 0.2 of the training dataset as a validation dataset. The corresponding model was learned by monitoring the validation loss and early stopping.

VIII. RESULTS AND ANALYSIS

We calculated feature importance which is ‘input perturbation’ [16], to understand the impact of each feature on the model. This method can determine the relative importance between features by evaluating the effect of small changes in the input of each feature on neural network output. For each feature, we calculated the difference between the model results for the original and noise data. The bigger the difference can be interpreted as a feature that has further affected the model. The top 10 were extracted after calculating the feature importance for the raw, cut, and pool methods. In fig. 4, *minionsKilled*, *ccReduction* were commonly selected as the most influential

features for distinguishing tiers. *minionsKilled* is a feature of how many minions were killed from the turrets. As a player kill a minion, the player can have gold to buy items in the game. As the number of gold increases, the items player can buy become more diverse so that the player can reach a strong champion early. *ccReduction* is a feature that expresses strength and is highly related to the player’s skills.

Area Under the Curve (AUC), an area below the Receiver Operating Characteristic (ROC) expressed as a measurement value from 0 to 1, is used as a performance metric. The ROC curve is a measurement graph of the model’s classification performance showing the True Positive Rate (TPR) value for the False Positive Rate (FPR). The results of three random experiments are shown in the table IV. We measure performance over the lapse of playtime to test the possibility of early detection. fig. 5 shows the value corresponding to the error range of 0.0005 and the elapsed time after the start of the match according to each data construction method. In the raw method, it should be 25 minutes to achieve the minimum 0.8271 AUC, and in the pool method, it should be 24 minutes to achieve the minimum 0.8768 AUC. Finally, in the cut method with the best performance 0.9036 AUC, the tier group could be the most accurately classified from the 24 minutes after the match starts. Through this, it is possible to preemptively respond to malicious behavior that may occur due to rank-skill mismatch before the match is over.

IX. DISCUSSION

Data processing is essential in deep learning models. We measured the performance by varying the dataset form, the model’s input, and selecting features according to the purpose of classifying tier groups. In the case of datasets, the raw method, which is most processable, did not fully utilize the batch size of the deep learning model, showing poor performance compared to other methods. The cut method showed the best performance compared to other methods, although there was data loss by removing data less than a certain length. The pool method, which needs more time for preprocessing but reduces data loss, produced results comparable to those of the cut method.

Feature selection is significant. By selecting features, computing power and memory can be used efficiently, mitigating

TABLE IV: The AUC performance results based on the three training dataset and the selected features

	original	remove 5	remove 9	remove 13	remove 18	remove 22
raw	0.8233	0.8200	0.8233	0.8100	0.8000	0.8233
cut	0.8966	0.8933	0.8966	0.8966	0.8833	0.8866
pool	0.8766	0.8766	0.8766	0.8733	0.8566	0.8633

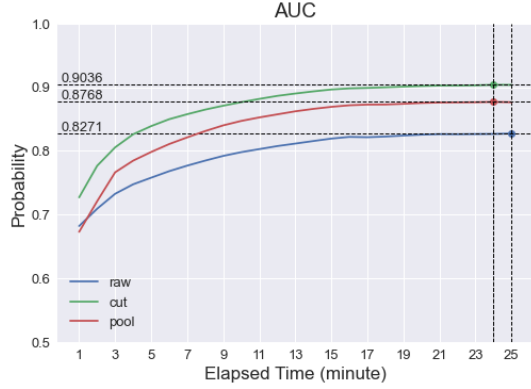


Fig. 5: The AUC value over time of the model. As the increase rate is less than 0.0005, we determine the value as the approximate maximum.

the overfitting of models and improving performance. We propose a feature removal scheme. In addition, it was experimentally verified that the performance of the model was maintained or very low by partially selecting features according to the method. As an existing feature selection method, there is a method of removing highly correlated features. When the model has the form of linear regression, features with similar characteristics are eliminated to leave only independent variables. As a representative tool, multicollinearity can be grasped through Variance Inflation Factors (VIF). Another existing feature selection method is to select features with significant influence through the feature importance of the model that requires enormous computing power.

Finally, we could identify which characteristics of players are highly relevant in classifying their skills through play patterns through feature importance. In order to create a model of higher intelligence, it is necessary to know which characteristics have a positive or negative effect on the outcome. The correlation is not affected by controlling one of the two factors, but the causal relationship is influenced by the other because the two factors have a precedence relationship. If causative variables can be identified through causal inference, the model can leap to a higher-intelligence model.

X. CONCLUSION

We propose the tier-group detection model to detect rank-skill mismatch that badly affect fair gameplay. It classified highly skilled players by analyzing behavior patterns in the player's timeline log using a time-series deep learning method. The model distinguished the skills of a professional gamer play pattern with the average performance of 0.8966 AUC in a cut

dataset. The raw method's dataset, which processes a variable length without pretreatment, was 0.8233 AUC, and the cut or pool method showed better results than the raw method. The feature *minionsKilled* and *ccReduction* were critical in all cases. The model showed 0.9036 AUC performance just 24 minutes after the start of the game; early detection will be possible. We obtained these results by selecting features with time series characteristics for each tier in the timeline match data and designing a model based on CNN-LSTM, a time series deep learning model, to analyze the data.

REFERENCES

- [1] Eoin Conroy, Magdalena Kowal, Adam J Toth, and Mark J Campbell. Boosting: Rank and skill deception in esports. *Entertainment Computing*, 36:100393, 2021.
- [2] Yubo Kou. Toxic behaviors in team-based competitive gaming. 2020.
- [3] League of legends official site, 2022.
- [4] Kuan-Ta Chen and Li-Wen Hong. User identification based on game-play activity patterns. In *Proceedings of the 6th ACM SIGCOMM workshop on Network and system support for games*, pages 7–12, 2007.
- [5] Ryan Kaminsky, Miro Enev, and Erik Andersen. Identifying game players with mouse biometrics. *University of Washington. Technical Report*, 2008.
- [6] Siming Liu, Christopher Ballinger, and Sushil J Louis. Player identification from rts game replays. *Proceedings of the 28th CATA*, pages 313–317, 2013.
- [7] Valmiro Ribeiro da Silva and Marjory Da Costa-Abreu. An empirical biometric-based study for user identification with different neural networks in the online game league of legends. In *2018 International Joint Conference on Neural Networks (IJCNN)*, pages 1–6. IEEE, 2018.
- [8] Soumik Mondal and Patrick Bours. Combining keystroke and mouse dynamics for continuous user authentication and identification. In *2016 IEEE International Conference on Identity, Security and Behavior Analysis (ISBA)*, pages 1–8. IEEE, 2016.
- [9] Chao Shen, Zhongmin Cai, Xiaohong Guan, and Roy Maxion. Performance evaluation of anomaly-detection algorithms for mouse dynamics. *computers & security*, 45:156–171, 2014.
- [10] Danial Hooshyar, Moslem Yousefi, and Heuiseok Lim. Data-driven approaches to game player modeling: a systematic literature review. *ACM Computing Surveys (CSUR)*, 50(6):1–19, 2018.
- [11] Yun-En Liu, Travis Mandel, Eric Butler, Erik Andersen, Eleanor O'Rourke, Emma Brunskill, and Zoran Popovic. Predicting player moves in an educational game: A hybrid approach. In *EDM*, pages 106–113. Citeseer, 2013.
- [12] Abhijeet Krishnan, Aaron Williams Aaron Williams, and Chris Martens. Towards action model learning for player modeling. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, volume 16, pages 238–244, 2020.
- [13] Jin Wang, Liang-Chih Yu, K Robert Lai, and Xuejie Zhang. Dimensional sentiment analysis using a regional cnn-lstm model. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 225–230, 2016.
- [14] Tae-Young Kim and Sung-Bae Cho. Predicting residential energy consumption using cnn-lstm neural networks. *Energy*, 182:72–81, 2019.
- [15] Ioannis E Livieris, Niki Kiriakidou, Stavros Stavroyiannis, and Panagiotis Pintelas. An advanced cnn-lstm model for cryptocurrency forecasting. *Electronics*, 10(3):287, 2021.
- [16] Michele Scardi and Lawrence W Harding Jr. Developing an empirical model of phytoplankton primary production: a neural network case study. *Ecological modelling*, 120(2-3):213–223, 1999.